

Chapter 11 — SQL Functions

****MySQL focus:**** MySQL 8.x / 8.4

****Goal:**** Learn how SQL functions transform, calculate, format, inspect, and summarize data inside queries.

11.1 What Is a SQL Function?

A SQL function is a built-in or user-defined operation that accepts one or more values (arguments) and returns a value or result.

General form:

```
FUNCTION_NAME(argument1, argument2, ...);
```

Example:

```
SELECT UPPER('hello');
```

Result:

```
HELLO
```

Functions are commonly used inside:

```
SELECT  
WHERE  
ORDER BY  
HAVING  
GROUP BY  
UPDATE  
INSERT
```

A function usually operates on a value/expression rather than changing the table definition.

Function vs SQL command

`SELECT`, `INSERT`, `UPDATE`, and `DELETE` are SQL statements/commands.

`UPPER()`, `ROUND()`, `COUNT()`, `COALESCE()`, and `DATE\_FORMAT()` are functions used inside expressions.

11.2 Types of Functions Covered in This Chapter

This chapter covers the major MySQL built-in function families needed for practical SQL and interviews:

1. String functions
2. Numeric and mathematical functions
3. Date and time functions
4. Aggregate functions
5. NULL-handling functions
6. Conditional / flow-control functions
7. Conversion functions
8. Information / system functions
9. JSON functions
10. Regular-expression functions — practical introduction
11. Combining multiple functions
12. Functions inside SELECT, WHERE, ORDER BY, GROUP BY and HAVING
13. Scalar vs aggregate functions

Deliberate chapter boundaries

The following are **not** taught in depth here **because** they deserve separate chapters:

- Detailed `GROUP BY` and `HAVING`
- JOINS
- Subqueries
- Set operators
- Views
- Indexes
- Transactions and ACID
- Window functions
- Stored procedures, stored functions, and triggers
- CTEs

They may appear only in a short context example where necessary.

11.3 Scalar vs Aggregate Functions

Scalar functions

A scalar function normally works on each input row independently and returns one value for that row.

Examples:

```
UPPER(student_name)
ROUND(salary, 2)
YEAR(join_date)
COALESCE(phone, 'Not Provided')
```

Example:

```
SELECT student_name, UPPER(student_name)
FROM students;
```

Each row gets its own transformed value.

Aggregate functions

Aggregate functions process a set of rows and return a summary value.

Examples:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

Example:

```
SELECT AVG(salary)
FROM employees;
```

The result is one summary value for the selected rows.

Important distinction

```
Scalar function    → row-level transformation
Aggregate function → group/set-level summary
```

11.4 String Functions

String functions are used to clean, format, search, combine, and extract text.

11.4.1 UPPER()

Converts text to uppercase.

```
SELECT UPPER('mysql');

SELECT UPPER(student_name) AS student_name_upper
FROM students;
```

11.4.2 LOWER()

Converts text to lowercase.

```
SELECT LOWER('MYSQL');
```

11.4.3 LENGTH()

Returns the length of a string in bytes.

```
SELECT LENGTH('Rameez');
```

For multibyte character sets, `CHAR\_LENGTH()` is often more appropriate when you want the number of characters.

11.4.4 CHAR_LENGTH()

Returns the number of characters.

```
SELECT CHAR_LENGTH('Rameez');
```

Interview point

`LENGTH()` counts bytes, while `CHAR\_LENGTH()` counts characters.

11.4.5 CONCAT()

Combines strings.

```
SELECT CONCAT('Shaik', ' ', 'Rameez');
```

Practical example:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

11.4.6 CONCAT_WS()

Combines values using a separator.

```
SELECT CONCAT_WS(' - ', 'Java', 'SQL', 'React');
```

`WS` means "with separator".

11.4.7 SUBSTRING()

Extracts part of a string.

```
SELECT SUBSTRING('DATABASE', 1, 4);
```

11.4.8 LEFT()

Returns characters from the left.

```
SELECT LEFT('DATABASE', 4);
```

11.4.9 RIGHT()

Returns characters from the right.

```
SELECT RIGHT('DATABASE', 4);
```

11.4.10 TRIM()

Removes leading and trailing spaces.

```
SELECT TRIM(' SQL ');
```

Related functions:

```
LTRIM()  
RTRIM()
```

11.4.11 REPLACE()

Replaces occurrences of text.

```
SELECT REPLACE('I like Java', 'Java', 'SQL');
```

Practical:

```
SELECT REPLACE(email, '@old.com', '@new.com')  
FROM customers;
```

11.4.12 REVERSE()

Reverses a string.

```
SELECT REVERSE('SQL');
```

11.4.13 LOCATE()

Finds the position of a substring.

```
SELECT LOCATE('SQL', 'I am learning SQL');
```

Related functions:

```
POSITION()  
INSTR()
```

11.4.14 LPAD() and RPAD()

Pads a string to a specified length.

```
SELECT LPAD('123', 5, '0');  
SELECT RPAD('123', 5, '0');
```

11.5 Numeric and Mathematical Functions

Numeric functions perform calculations and mathematical transformations.

11.5.1 ABS()

Returns absolute value.

```
SELECT ABS(-25);
```

11.5.2 ROUND()

Rounds a number.

```
SELECT ROUND(123.4567, 2);
```

11.5.3 CEIL() / CEILING()

Rounds upward to the next integer.

```
SELECT CEIL(12.1);
```

11.5.4 FLOOR()

Rounds downward.

```
SELECT FLOOR(12.9);
```

11.5.5 TRUNCATE()

Removes decimal places without normal rounding.

```
SELECT TRUNCATE(123.4567, 2);
```

ROUND vs TRUNCATE

```
ROUND(123.456, 2)    → 123.46
TRUNCATE(123.456, 2) → 123.45
```

11.5.6 MOD() and %

Returns the remainder.

```
SELECT MOD(10, 3);
SELECT 10 % 3;
```

Useful for even/odd checks:

```
SELECT student_id
FROM students
WHERE MOD(student_id, 2) = 0;
```

11.5.7 POWER()

Raises a number to a power.

```
SELECT POWER(2, 3);
```

11.5.8 SQRT()

Returns square root.

```
SELECT SQRT(64);
```

11.5.9 SIGN()

Returns the sign of a number.

```
SELECT SIGN(-10);
SELECT SIGN(0);
SELECT SIGN(10);
```

11.5.10 GREATEST() and LEAST()

Compare multiple expressions.

```
SELECT GREATEST(10, 25, 15);
SELECT LEAST(10, 25, 15);
```

11.5.11 RAND()

Generates a pseudo-random value.

```
SELECT RAND();
```

Be careful using non-deterministic functions when reproducible results are required.

11.6 Date and Time Functions

Date/time functions are heavily used in real applications such as orders, payments, employee joining dates, subscriptions, and reports.

11.6.1 NOW()

Returns current date and time.

```
SELECT NOW();
```

11.6.2 CURRENT_DATE()

Returns current date.

```
SELECT CURRENT_DATE();
```

11.6.3 CURRENT_TIME()

Returns current time.

```
SELECT CURRENT_TIME();
```

11.6.4 CURRENT_TIMESTAMP()

Returns current timestamp.

```
SELECT CURRENT_TIMESTAMP();
```

11.6.5 DATE()

Extracts the date portion.

```
SELECT DATE('2026-09-17 18:30:00');
```

11.6.6 TIME()

Extracts the time portion.

```
SELECT TIME('2026-09-17 18:30:00');
```

11.6.7 YEAR(), MONTH(), DAY()

Extract date components.

```
SELECT YEAR(join_date)
FROM employees;

SELECT MONTH(join_date)
FROM employees;

SELECT DAY(join_date)
FROM employees;
```

11.6.8 MONTHNAME() and DAYNAME()

Return readable names.

```
SELECT MONTHNAME(join_date)
FROM employees;

SELECT DAYNAME(join_date)
FROM employees;
```

11.6.9 HOUR(), MINUTE(), SECOND()

Extract time components.

```
SELECT HOUR(login_time)
FROM login_history;
```

11.6.10 DATE_ADD()

Adds an interval.

```
SELECT DATE_ADD('2026-09-17', INTERVAL 7 DAY);
```

Example:

```
SELECT DATE_ADD(join_date, INTERVAL 1 YEAR)
FROM employees;
```

11.6.11 DATE_SUB()

Subtracts an interval.

```
SELECT DATE_SUB('2026-09-17', INTERVAL 30 DAY);
```

11.6.12 DATEDIFF()

Returns the number of days between two dates.

```
SELECT DATEDIFF('2026-09-17', '2026-09-01');
```

Important: `DATEDIFF()` focuses on the date difference in days.

11.6.13 TIMESTAMPDIFF()

Calculates a difference using a selected unit.

```
SELECT TIMESTAMPDIFF(
    YEAR,
    '2005-01-01',
    '2026-01-01'
);
```

Useful for age or service-period calculations.

11.6.14 DATE_FORMAT()

Formats a date/time value.

```
SELECT DATE_FORMAT('2026-09-17', '%d-%m-%Y');
```

Common format specifiers:

```
%Y → four-digit year
%y → two-digit year
%m → month number
%d → day
%H → hour
%i → minute
%s → second
```

11.7 Aggregate Functions

Aggregate functions summarize multiple rows.

11.7.1 COUNT()

Counts rows or non-NULL values depending on the expression.

```
SELECT COUNT(*)
FROM employees;

SELECT COUNT(phone)
FROM employees;
```

`COUNT(\*)` counts rows; `COUNT(column)` counts non-NULL values in that column.

11.7.2 SUM()

Adds numeric values.

```
SELECT SUM(salary)
FROM employees;
```

11.7.3 AVG()

Returns average.

```
SELECT AVG(salary)
FROM employees;
```

11.7.4 MIN()

Returns minimum.

```
SELECT MIN(salary)
FROM employees;
```

11.7.5 MAX()

Returns maximum.

```
SELECT MAX(salary)
FROM employees;
```

11.7.6 Aggregate functions and NULL

Aggregate functions generally ignore NULL values for the input column.

Example:

```
SELECT AVG(salary)
FROM employees;
```

Rows where `salary` is NULL are not treated as numeric salary values.

11.7.7 Aggregate functions with GROUP BY

Basic relationship:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

This chapter introduces the function aspect only.

****Detailed `GROUP BY`, grouping rules, `HAVING`, grouping errors, and advanced aggregation will be covered in a separate chapter.****

11.8 NULL-Handling Functions

NULL means a value is missing/unknown; it is not the same as zero or an empty string.

11.8.1 IFNULL()

Returns an alternative value when the first expression is NULL.

```
SELECT IFNULL(phone, 'Not Provided')
FROM customers;
```

11.8.2 COALESCE()

Returns the first non-NULL expression.

```
SELECT COALESCE(phone, email, 'No Contact')
FROM customers;
```

This is especially useful when several fallback values are possible.

11.8.3 NULLIF()

Returns NULL if two expressions are equal; otherwise returns the first expression.

```
SELECT NULLIF(10, 10);
SELECT NULLIF(10, 20);
```

A common use is preventing division by zero:

```
SELECT sales / NULLIF(quantity, 0)
FROM products;
```

11.9 Conditional / Flow-Control Functions

These functions allow conditional logic inside expressions.

11.9.1 IF()

Syntax:

```
IF(condition, value_if_true, value_if_false)
```

Example:

```
SELECT employee_name,
       IF(salary >= 50000, 'High', 'Normal') AS salary_level
FROM employees;
```

11.9.2 CASE

`CASE` supports multiple conditions and is usually more readable for complex business rules.

```
SELECT employee_name,  
       CASE  
         WHEN salary >= 100000 THEN 'Senior'  
         WHEN salary >= 50000 THEN 'Mid-Level'  
         ELSE 'Junior'  
       END AS level  
FROM employees;
```

Simple CASE

```
CASE department  
  WHEN 'IT' THEN 'Technology'  
  WHEN 'HR' THEN 'People'  
  ELSE 'Other'  
END
```

Searched CASE

```
CASE  
  WHEN salary >= 100000 THEN 'Senior'  
  WHEN salary >= 50000 THEN 'Mid-Level'  
  ELSE 'Junior'  
END
```

11.10 Conversion Functions

Conversion functions change or interpret a value as another data type.

11.10.1 CAST()

General form:

```
CAST(expression AS data_type)
```

Example:

```
SELECT CAST('2026-09-17' AS DATE);
```

Numeric example:

```
SELECT CAST('125.50' AS DECIMAL(10,2));
```

Common interview idea:

CAST is useful when an expression needs to be treated as a specific data type.

11.10.2 CONVERT()

MySQL also provides `CONVERT()` for conversion.

```
SELECT CONVERT('125', UNSIGNED);
```

It can also be used for character-set conversion.

11.11 Information / System Functions

These functions return information about the MySQL environment or current session.

VERSION()

```
SELECT VERSION();
```

DATABASE()

Returns the currently selected database.

```
SELECT DATABASE();
```

USER()

Returns the current MySQL account information.

```
SELECT USER();
```

CONNECTION_ID()

Returns the current connection/session ID.

```
SELECT CONNECTION_ID();
```

These are useful for troubleshooting and understanding the current database session.

11.12 JSON Functions — Practical Introduction

MySQL supports JSON data and provides functions for creating, reading, validating, and working with JSON values.

JSON is semi-structured data and is especially common in APIs and modern applications.

11.12.1 JSON_OBJECT()

Creates a JSON object.

```
SELECT JSON_OBJECT(
    'name', 'Rameez',
    'skill', 'SQL'
);
```

11.12.2 JSON_ARRAY()

Creates a JSON array.

```
SELECT JSON_ARRAY('Java', 'SQL', 'React');
```

11.12.3 JSON_EXTRACT()

Extracts a value using a JSON path.

```
SELECT JSON_EXTRACT(
    '{"name": "Rameez", "age": 21}',
    '$.name'
);
```

11.12.4 JSON_UNQUOTE()

Removes JSON string quotes from an extracted JSON string.

```
SELECT JSON_UNQUOTE(
    JSON_EXTRACT(
        '{"name": "Rameez"}',
        '$.name'
    )
);
```

11.12.5 JSON_VALID()

Checks whether a value is valid JSON.

```
SELECT JSON_VALID('{"name": "Rameez"}');
```

JSON scope boundary

This chapter covers common JSON functions needed to understand SQL/JSON interaction.

Advanced JSON topics such as `JSON\_TABLE()`, JSON schema validation, generated columns, and JSON indexing belong in an advanced chapter.

11.13 Regular Expression Functions — Practical Introduction

MySQL provides regular-expression functionality for advanced text matching.

The `REGEXP` / `RLIKE` operators can test whether a string matches a pattern.

Example:

```
SELECT email
FROM customers
WHERE email REGEXP '^[A-Za-z0-9._%+-]+@';
```

Use regular expressions when ordinary `LIKE` patterns are not expressive enough.

LIKE vs REGEXP

```
LIKE    → simple wildcard matching
REGEXP  → pattern-based matching
```

Detailed regular-expression syntax is outside the main scope of this chapter and can be expanded later if required.

11.14 Functions Inside SELECT

Functions are frequently used to transform output.

```
SELECT
  employee_name,
  UPPER(employee_name) AS name_upper,
  ROUND(salary, 2) AS rounded_salary
FROM employees;
```

11.15 Functions Inside WHERE

Functions can be used to filter rows.

```
SELECT *
FROM employees
WHERE UPPER(department) = 'IT';
```

Date example:

```
SELECT *
FROM employees
WHERE YEAR(join_date) = 2026;
```

Performance awareness

Applying a function to a column in a filter can affect index usage in some situations. For large production tables, prefer a predicate that can use the stored value directly when practical.

11.16 Functions Inside ORDER BY

```
SELECT employee_name, salary
FROM employees
ORDER BY ROUND(salary, -3) DESC;
```

You can also sort using a selected expression/alias where supported.

11.17 Functions Inside GROUP BY

A function can create the grouping expression.

```
SELECT YEAR(join_date) AS join_year,  
       COUNT(*) AS employee_count  
FROM employees  
GROUP BY YEAR(join_date);
```

This chapter focuses on the function expression itself.

Detailed grouping rules are reserved for the dedicated GROUP BY/HAVING chapter.

11.18 Functions Inside HAVING

Aggregate functions are commonly used with `HAVING`.

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 50000;
```

The example is included to show where aggregate functions appear.

Detailed `HAVING` behavior is reserved for the GROUP BY/HAVING chapter.

11.19 Combining Functions

Real SQL often combines multiple functions.

Example:

```
SELECT  
    UPPER(TRIM(employee_name)) AS cleaned_name,  
    ROUND(salary * 1.10, 2) AS revised_salary  
FROM employees;
```

Date + formatting:

```
SELECT DATE_FORMAT(join_date, '%d-%m-%Y') AS formatted_date  
FROM employees;
```

NULL + string:

```
SELECT CONCAT(  
    employee_name,  
    ' - ',  
    COALESCE(department, 'Unknown')  
) AS employee_info  
FROM employees;
```

Conditional + aggregate:

```
SELECT  
    COUNT(IF(status = 'Active', 1, NULL)) AS active_count  
FROM employees;
```

11.20 Important NULL Behavior

A common mistake is assuming functions always return a useful value when one argument is NULL.

Many expressions involving NULL produce NULL unless the function explicitly handles NULL.

Example:

```
SELECT CONCAT('Hello ', NULL);
```

Use NULL-handling functions when needed:

```
SELECT CONCAT('Hello ', COALESCE(name, 'Guest'));
```

Always check the documented NULL behavior of a function.

11.21 Common Function Mistakes

Mistake 1 — Confusing LENGTH and CHAR_LENGTH

LENGTH → bytes
CHAR_LENGTH → characters

Mistake 2 — Confusing ROUND and TRUNCATE

ROUND → rounds
TRUNCATE → cuts decimal places

Mistake 3 — Treating NULL as zero

NULL is not automatically zero.

Mistake 4 — Using `= NULL`

Wrong:

```
WHERE phone = NULL
```

Correct:

```
WHERE phone IS NULL
```

Mistake 5 — Confusing COUNT(*) and COUNT(column)

COUNT(*) → counts rows
COUNT(column) → counts non-NULL values in that column

Mistake 6 — Using an aggregate function as if it were row-level

`AVG(salary)` summarizes a set; `ROUND(salary, 2)` transforms each salary value.

Mistake 7 — Forgetting DATE_FORMAT specifiers

The format string is not arbitrary text; `%Y`, `%m`, `%d`, etc. have specific meanings.

11.22 Practical SQL Problems

Problem 1 — Clean employee names

Display employee names in uppercase after removing extra spaces.

Concepts:

TRIM
UPPER

Problem 2 — Calculate annual salary

Display employee name and annual salary from monthly salary.

Concept:

Arithmetic expression

Problem 3 — Round product prices

Display product prices rounded to two decimal places.

Concept:

ROUND

Problem 4 — Find employees who joined in 2026

Use a date function to filter employees.

Concept:

YEAR

Problem 5 — Calculate years of service

Calculate the number of years between joining date and the current date.

Concepts:

TIMESTAMPDIFF
CURRENT_DATE

Problem 6 — Replace missing phone numbers

Display `Not Provided` when a phone number is NULL.

Concept:

COALESCE / IFNULL

Problem 7 — Create a full name

Combine first name and last name.

Concept:

CONCAT

Problem 8 — Categorize salaries

Classify employees as Junior, Mid-Level, or Senior.

Concept:

CASE

Problem 9 — Count employees

Find the total number of employees.

Concept:

COUNT

Problem 10 — Find average salary

Find the average salary of all employees.

Concept:

AVG

Problem 11 — Find highest and lowest salary

Use:

MAX
MIN

Problem 12 — Find departments with average salary above a threshold

Use:

AVG
GROUP BY
HAVING

This is a bridge problem; detailed grouping is covered later.

Problem 13 — Extract JSON information

Given a JSON column containing employee metadata, extract the employee's city.

Concept:

JSON_EXTRACT

11.23 Interview Questions

Q1. What is a SQL function?

A SQL function is an operation that accepts values/expressions and returns a value or summary result.

Q2. What is the difference between scalar and aggregate functions?

Scalar functions normally return a value for each input row; aggregate functions summarize a set of rows into a result.

Q3. What is the difference between LENGTH() and CHAR_LENGTH()?

`LENGTH()` returns the number of bytes, while `CHAR\_LENGTH()` returns the number of characters.

Q4. Difference between ROUND() and TRUNCATE()?

`ROUND()` applies rounding; `TRUNCATE()` removes digits beyond the specified decimal position without normal rounding.

Q5. Difference between COUNT(*) and COUNT(column)?

`COUNT(\*)` counts rows. `COUNT(column)` counts non-NULL values in that column.

Q6. What does COALESCE() do?

It returns the first non-NULL expression from its arguments.

Q7. IFNULL() vs COALESCE()?

`IFNULL()` handles a two-value fallback. `COALESCE()` can evaluate multiple expressions and returns the first non-NULL one.

Q8. Why use NULLIF()?

It can turn a matching value into NULL and is commonly used to avoid division by zero.

Q9. What is CASE used for?

`CASE` implements conditional logic inside SQL expressions.

Q10. What is DATE_FORMAT() used for?

It formats date/time values according to MySQL format specifiers.

Q11. What is the difference between DATEDIFF() and TIMESTAMPDIFF()?

`DATEDIFF()` returns a difference in days; `TIMESTAMPDIFF()` lets you specify a unit such as YEAR, MONTH, DAY, HOUR, etc.

Q12. Can functions be used in WHERE?

Yes. Functions can be used inside expressions in `WHERE`.

Q13. Can functions be used in ORDER BY?

Yes.

Q14. Can aggregate functions be used with GROUP BY?

Yes. Aggregate functions are commonly used to summarize each group.

Q15. What happens to NULL in aggregate functions?

Many aggregate functions ignore NULL input values for the aggregated expression.

Q16. What is CAST()?

`CAST()` converts an expression to a specified data type.

Q17. Why are JSON functions useful?

They allow SQL queries to create, inspect, extract, and work with JSON values stored or produced by applications.

Q18. LIKE vs REGEXP?

`LIKE` provides simpler wildcard matching; `REGEXP` supports regular-expression pattern matching.

11.24 Chapter 11 Function Cheat Sheet

| Category | Important Functions |

|---|---|

| String | UPPER, LOWER, LENGTH, CHAR_LENGTH, CONCAT, CONCAT_WS, SUBSTRING, LEFT, RIGHT, TRIM, LTRIM, RTRIM, REPLACE, REVERSE, LOCATE, POSITION, INSTR, LPAD, RPAD |

| Numeric | ABS, ROUND, CEIL, CEILING, FLOOR, TRUNCATE, MOD, %, POWER, SQRT, SIGN, GREATEST, LEAST, RAND |

| Date/Time | NOW, CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, DATE, TIME, YEAR, MONTH, MONTHNAME, DAY, DAYNAME, HOUR, MINUTE, SECOND, DATE_ADD, DATE_SUB, DATEDIFF, TIMESTAMPDIFF, DATE_FORMAT |

| Aggregate | COUNT, SUM, AVG, MIN, MAX |

| NULL | IFNULL, COALESCE, NULLIF |

| Conditional | IF, CASE |

| Conversion | CAST, CONVERT |

| Information | VERSION, DATABASE, USER, CONNECTION_ID |

| JSON | JSON_OBJECT, JSON_ARRAY, JSON_EXTRACT, JSON_UNQUOTE, JSON_VALID |

| Pattern matching | REGEXP, RLIKE |

11.25 What Comes Next?

Chapter 11 teaches **functions themselves**.

The next chapters can build on these functions without mixing unrelated concepts into this chapter:

- Chapter 11 → SQL Functions
- Chapter 12 → Aggregate Queries, GROUP BY & HAVING
- Chapter 13 → SQL JOINS
- Chapter 14 → Subqueries
- Chapter 15 → Set Operators
- Chapter 16 → CASE / Advanced Conditional SQL
- Chapter 17 → Views & Indexes
- Chapter 18 → Transactions & ACID
- Chapter 19 → Normalization & Database Design
- Chapter 20 → CTEs
- Chapter 21 → Window Functions
- Chapter 22 → Advanced SQL + MNC Interview Problems

Important: The exact future chapter numbering can be changed later. The key boundary is that Chapter 11 remains focused on SQL functions.

11.26 Final Summary

SQL functions make queries more useful by transforming values, performing calculations, handling dates, managing NULL values, applying conditions, converting data types, summarizing rows, and working with structured/semi-structured values such as JSON.

For interviews, remember the core groups:

- String
- Numeric
- Date/Time
- Aggregate
- NULL Handling
- Conditional
- Conversion
- Information/System
- JSON
- Pattern Matching

And remember the most important distinction:

- Scalar function → works row by row
- Aggregate function → summarizes multiple rows

MySQL's official documentation groups built-in functions into families including flow-control, numeric, date/time, string, conversion, information, JSON, aggregate, and window functions. This chapter intentionally teaches the practical function families while reserving large standalone topics such as joins, transactions, and window functions for later chapters.